Algorithms and Pseudocode

Python Activity-Based Learning — 10 Selected Case Studies

Use these as simple guide notes alongside the matching flowcharts and Python programs. Keep variable names and conditions consistent with the code you submitted.

Basic format

Algorithm: write the steps in order using simple sentences. Pseudocode: write the same logic using words such as INPUT, IF, ELSE, WHILE, FOR, DISPLAY and STOP.

1. Attendance Recovery Computation

Algorithm

1. Start.
2. Read the number of classes conducted, classes attended and target attendance percentage.
3. Check that the inputs are valid. If they are invalid, display an error and stop.
4. If no classes have been conducted, display that the current percentage cannot be calculated.
5. Calculate current attendance percentage.
6. Set future classes to 0.
7. While attendance after attending all future classes is below the target, increase future classes by 1.
8. Display current attendance, minimum future classes required and attendance after those classes.
9. Stop.

Pseudocode

```
START
INPUT conducted, attended, target
IF conducted < 0 OR attended < 0 OR attended > conducted
OR target < 0 OR target > 100 THEN
DISPLAY "Invalid input"
ELSE IF conducted = 0 THEN
DISPLAY "Attendance percentage unavailable"
ELSE
current = attended / conducted * 100
future = 0
WHILE ((attended + future) / (conducted + future) * 100) < target
future = future + 1
END WHILE
DISPLAY current, future
DISPLAY attendance after future classes
END IF
STOP
```

2. ATM Transaction Sequence Simulation

Algorithm

1. Start.

2. Read opening balance and number of transactions.

3. If the balance or transaction count is negative, display an error and stop.

4. Set accepted and rejected transaction counts to 0; create an empty record list.

5. For each transaction, read the type (D or W) and amount.

6. If the amount is not positive, reject it. Otherwise, accept a deposit, or accept a withdrawal only when the balance is enough. Reject any other transaction.

7. Update the balance and accepted/rejected count as appropriate; save the transaction details.

8. After all transactions, display the transaction records, final balance and counts.

9. Stop.

Pseudocode

```
START
INPUT balance, count
IF balance < 0 OR count < 0 THEN
DISPLAY "Invalid input"
ELSE
accepted = 0
rejected = 0
records = empty list
FOR each transaction from 1 to count
INPUT kind, amount
IF amount <= 0 THEN
status = "Rejected"
rejected = rejected + 1
ELSE IF kind = "D" THEN
balance = balance + amount
status = "Deposit accepted"
accepted = accepted + 1
ELSE IF kind = "W" AND amount <= balance THEN
balance = balance - amount
status = "Withdrawal accepted"
accepted = accepted + 1
ELSE
status = "Rejected"
rejected = rejected + 1
END IF
ADD kind, amount, status TO records
DISPLAY status
END FOR
DISPLAY records, balance, accepted, rejected
END IF
STOP
```

3. Warehouse Inventory Reconciliation

Algorithm

1. Start.

2. Read the number of products.

3. For each product, read its name, opening stock, received quantity, dispatched quantity and actual closing stock.

4. Calculate expected closing stock as opening stock + received quantity - dispatched quantity.

5. Compare expected closing stock with actual closing stock.

6. If they differ, calculate the difference and identify whether stock needs checking or replenishment; otherwise mark it as matched.

7. Repeat for the remaining products.

8. Display the product-wise stock status and summary.

9. Stop.

Pseudocode

```
START
INPUT number_of_products
FOR each product
INPUT name, opening, received, dispatched, actual_closing
expected = opening + received - dispatched
IF expected = actual_closing THEN
status = "Matched"
difference = 0
ELSE
difference = expected - actual_closing
status = "Discrepancy found"
END IF
DISPLAY name, expected, actual_closing, difference, status
END FOR
DISPLAY stock summary
STOP
```

4. Student Performance Trend Analysis

Algorithm

1. Start.
2. Prepare an empty list to store subject results.
3. For each subject, read the earlier mark and latest mark.
4. Calculate the change by subtracting the earlier mark from the latest mark.
5. Record whether the mark increased, decreased or stayed the same.
6. Repeat for all subjects.
7. Calculate the latest average and identify weak subjects according to the program's rule.
8. Display subject-wise changes, latest average, weak subjects and overall classification.
9. Stop.

Pseudocode

```
START
CREATE empty results list
FOR each subject
INPUT subject, first_mark, latest_mark
change = latest_mark - first_mark
IF change > 0 THEN
trend = "Improved"
ELSE IF change < 0 THEN
trend = "Declined"
ELSE
trend = "No change"
END IF
SAVE subject, change, trend
END FOR
CALCULATE latest average
IDENTIFY weak subjects
DECIDE overall classification using the program's rule
DISPLAY results, average, weak subjects, classification
STOP
```

5. Multi-Product Shopping Budget Constraint

Algorithm

1. Start.

2. Read the number of products and shopping budget.

3. Check that the count and budget are not negative; otherwise display an error.

4. Set subtotal to 0 and prepare lists for product names and item totals.

5. For each product, read its name, unit price and quantity. If price or quantity is negative, display an error and stop processing the bill.

6. Calculate item total and add it to the subtotal.

7. After all products, apply a 10% discount if the subtotal is at least 2000.

8. Compare the final total with the budget and display the amount over budget or the remaining balance.

9. Stop.

Pseudocode

```
START
INPUT n, budget
IF n < 0 OR budget < 0 THEN
DISPLAY "Invalid input"
ELSE
subtotal = 0
FOR each product from 1 to n
INPUT name, price, quantity
IF price < 0 OR quantity < 0 THEN
DISPLAY "Invalid price or quantity"
STOP
END IF
item_total = price * quantity
subtotal = subtotal + item_total
SAVE name and item_total
END FOR
IF subtotal >= 2000 THEN
discount = subtotal * 0.10
ELSE
discount = 0
END IF
final_total = subtotal - discount
IF final_total > budget THEN
DISPLAY "Over budget", final_total - budget
ELSE
DISPLAY "Remaining budget", budget - final_total
END IF
DISPLAY item totals, subtotal, discount, final_total
END IF
STOP
```

6. Nested Loop Examination Seating Generator

Algorithm

1. Start.
2. Read the number of rows, columns and student roll numbers.
3. If rows or columns are not positive, display an error and stop.
4. Create a seating layout with every seat initially empty.
5. Read the number of reserved seats and mark each valid reserved seat.
6. Go through the seating layout row by row and seat by seat.
7. If a seat is empty and students are still available, place the next student there.
8. After checking all seats, display the seating layout and any students who could not be seated.
9. Stop.

Pseudocode

```
START
INPUT rows, columns, student_list
IF rows <= 0 OR columns <= 0 THEN
DISPLAY "Rows and columns must be positive"
STOP
END IF
CREATE seating layout filled with empty markers
INPUT reserved_count
REPEAT reserved_count times
INPUT reserved_row, reserved_column
IF coordinates are within the layout THEN
MARK that seat as reserved
END IF
END REPEAT
index = 0
FOR each row
FOR each column
IF seat is empty AND index < number of students THEN
PLACE student_list[index] in seat
index = index + 1
END IF
END FOR
END FOR
DISPLAY seating layout
DISPLAY students from index onward as unallocated
STOP
```

7. String-Based Examination Answer Evaluation

Algorithm

1. Start.

2. Read the student's answers and the answer key.

3. Set correct, incorrect and blank counts to 0; set score to 0.

4. Compare each student's answer with the matching key answer, ignoring letter case.

5. If it matches, add 1 to the score and correct count. If blank, increase blank count. Otherwise subtract 0.25 and increase incorrect count.

6. Repeat until all questions have been checked.

7. Display the final score and counts.

8. Stop.

Pseudocode

```
START
INPUT student_answers, answer_key
correct = 0
incorrect = 0
blank = 0
score = 0
FOR each question
compare student answer with key, ignoring case
IF answer matches key THEN
correct = correct + 1
score = score + 1
ELSE IF answer is blank THEN
blank = blank + 1
ELSE
incorrect = incorrect + 1
score = score - 0.25
END IF
END FOR
DISPLAY score, correct, incorrect, blank
STOP
```

8. Text Compression Using Character Runs

Algorithm

1. Start.
2. Read the text.
3. If the text is empty, display an empty result and stop.
4. Set the current character to the first character and count it as 1.
5. Read the next character. If it matches the current character, increase the count.
6. If it is different, add the current character and its count to the result, then start a new count for the new character.
7. Repeat until the text ends.
8. Add the final character run to the result and display the compressed text.
9. Stop.

Pseudocode

```
START
INPUT text
IF text is empty THEN
DISPLAY empty result
STOP
END IF
current = first character
count = 1
result = empty
FOR each next character in text
IF character = current THEN
count = count + 1
ELSE
ADD current and count TO result
current = character
count = 1
END IF
END FOR
ADD current and count TO result
DISPLAY result
STOP
```

9. Student Marks Matrix Analysis

Algorithm

1. Start.
2. Read the marks for students and subjects into a matrix.
3. Prepare totals and other values needed for analysis.
4. For each student, add their subject marks and calculate the average.
5. Check the marks while processing the matrix to find the highest and lowest marks, if required.
6. Repeat until all students and subjects have been processed.
7. Display student totals, averages and the marks analysis.
8. Stop.

Pseudocode

```
START
INPUT marks matrix
SET totals and analysis values
FOR each student row
total = 0
FOR each subject mark in that row
total = total + mark
UPDATE highest and lowest marks if needed
END FOR
average = total / number of subjects
SAVE total and average
END FOR
DISPLAY totals, averages, highest and lowest marks
STOP
```

10. Food Waste Monitoring System

Algorithm

1. Start.
2. Read food prepared, food served and leftover amounts for each record.
3. Check that the values are valid for the program.
4. Calculate the waste amount and waste percentage.
5. Compare the waste percentage with the set limit and classify the record.
6. Repeat for the remaining records and update the summary values.
7. Display each record's result and the overall waste summary.
8. Stop.

Pseudocode

```
START
INPUT food records and waste limit
FOR each record
INPUT food_prepared, food_served, leftover
CHECK input values
waste = leftover
IF food_prepared > 0 THEN
waste_percent = waste / food_prepared * 100
ELSE
waste_percent = 0
END IF
IF waste_percent > waste_limit THEN
status = "Above limit"
ELSE
status = "Within limit"
END IF
UPDATE summary values
DISPLAY record result
END FOR
DISPLAY overall waste summary
STOP
```